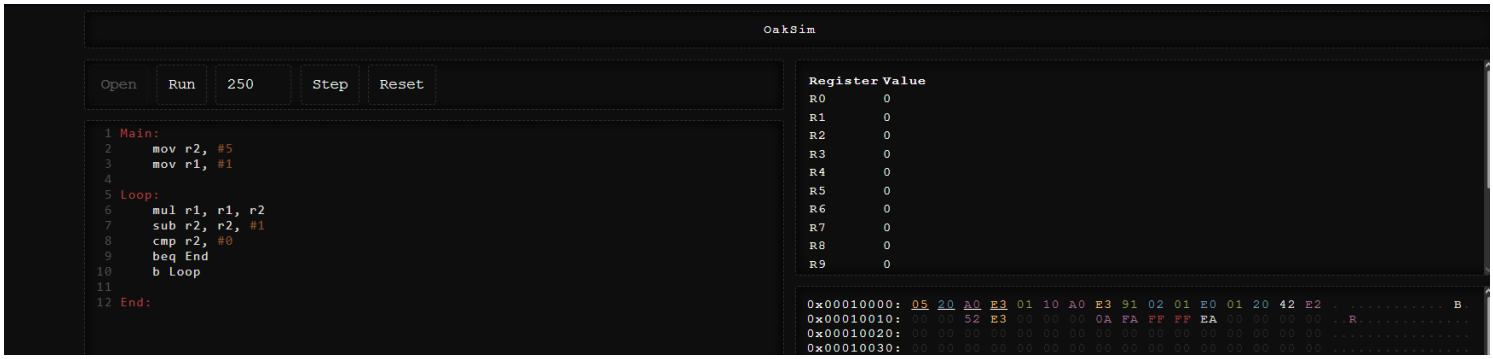
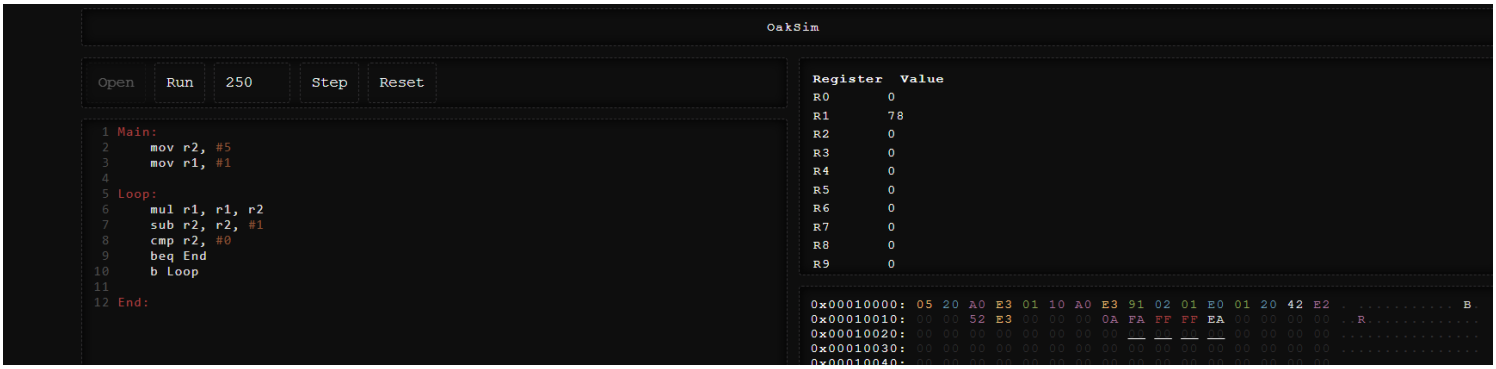
Template Week 4 – Software

Student number:530010

Assignment 4.1: ARM assembly

Screenshot of working assembly code of factorial calculation:

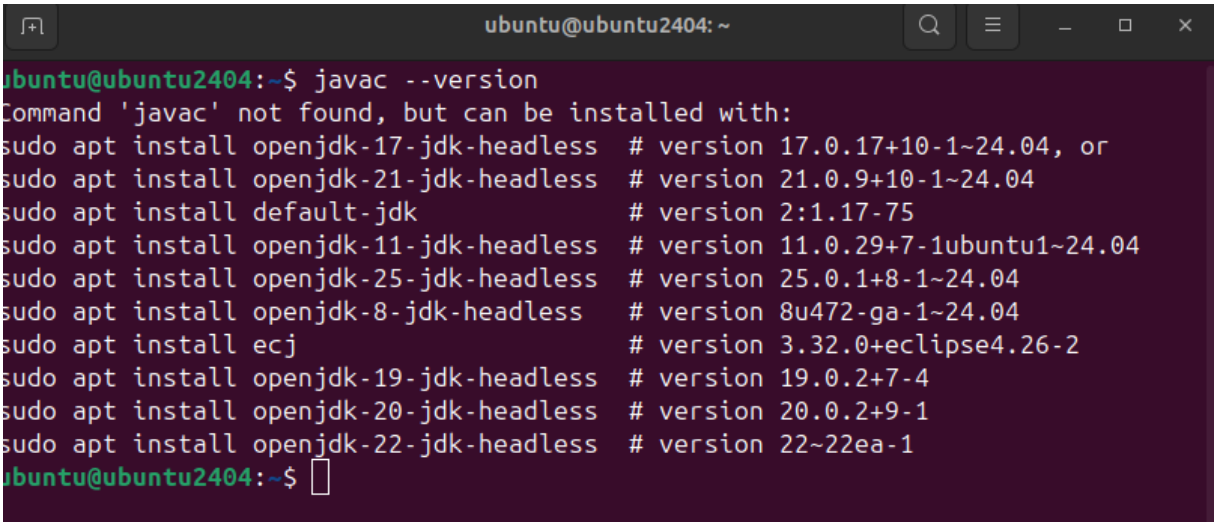




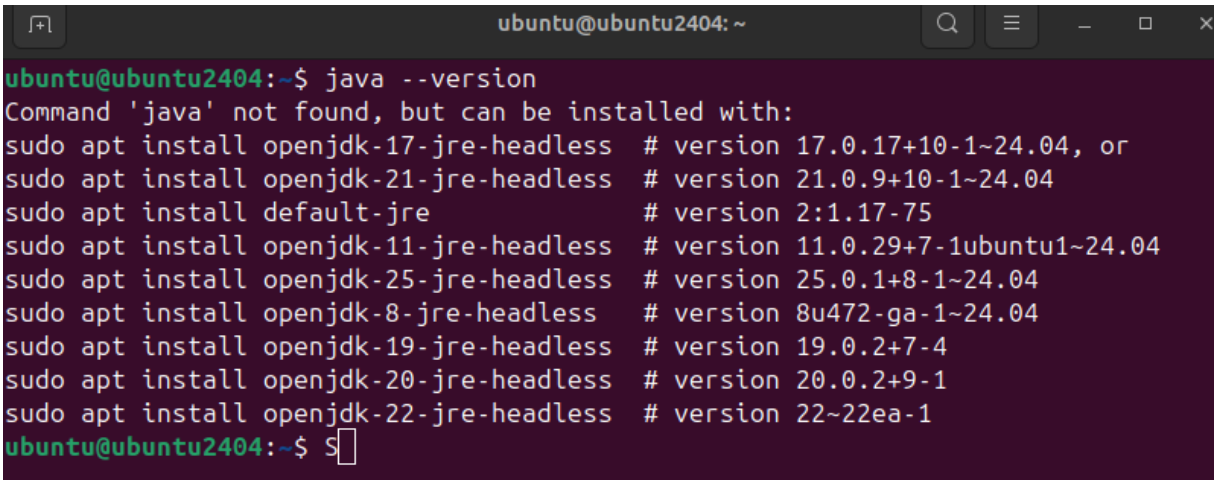
Assignment 4.2: Programming languages

Take screenshots that the following commands work:

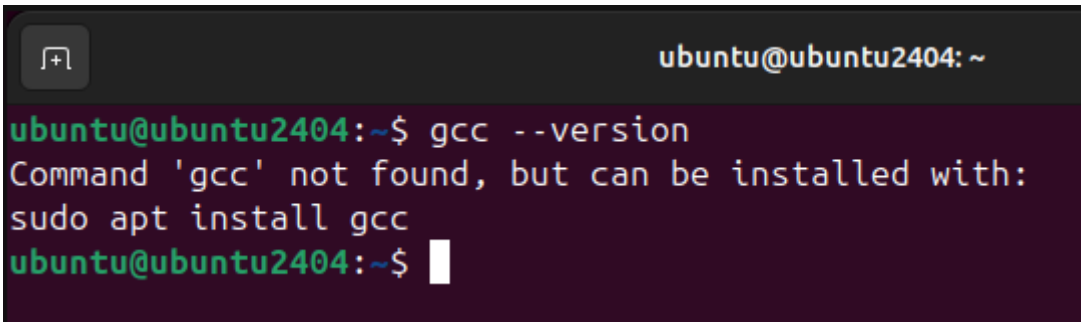
javac –version



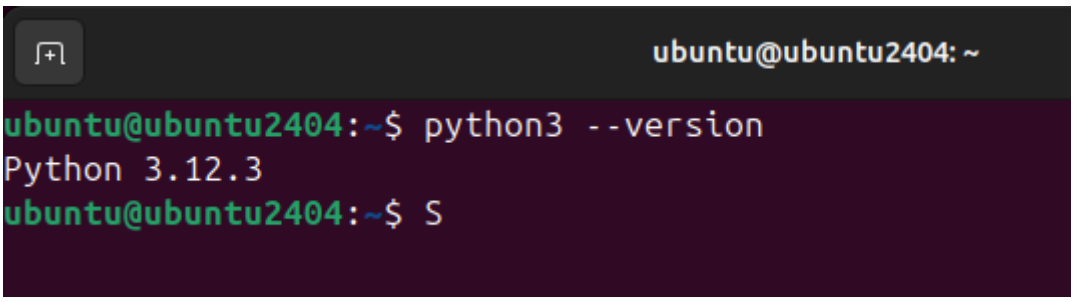
java –version



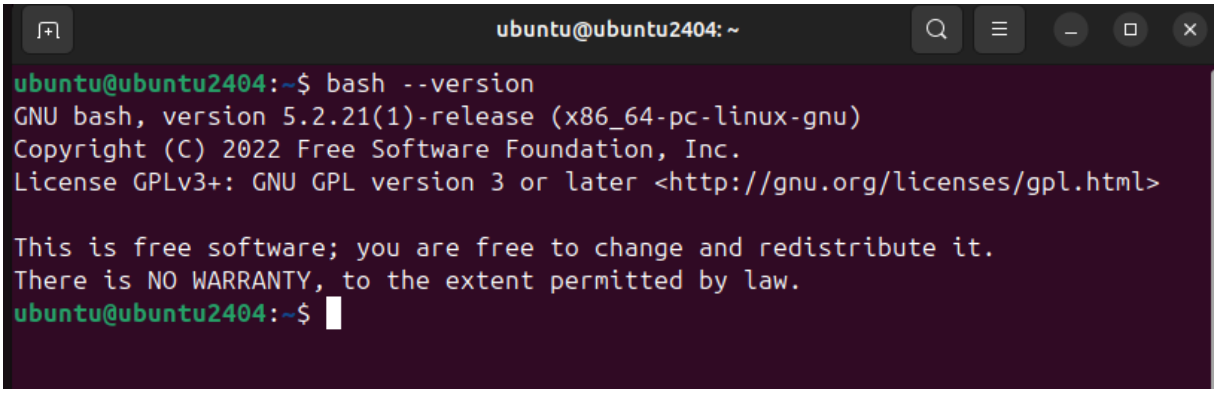
gcc –version



python3 –version



bash –version



Assignment 4.3: Compile

Which of the above files need to be compiled before you can run them?

Fib.c en fib.java

Which source code files are compiled into machine code and then directly executable by a processor?

Fib.c

Which source code files are compiled to byte code?

Fib.java

Which source code files are interpreted by an interpreter?

Fib.py en fib.sh

These source code files will perform the same calculation after compilation/interpretation. Which one is expected to do the calculation the fastest?

Fib.c

How do I run a Java program?

Javac Program.java java Program

How do I run a Python program?

Python program.py

How do I run a C program?

Gcc program.c -o program ./program

How do I run a Bash script?

bash script.sh

If I compile the above source code, will a new file be created? If so, which file?

Fib.c = yes = Executable (e.g., program)

Java = yes = Bytecode (program.class)

Python = no

Bash = no

Take relevant screenshots of the following commands:

- Compile the source files where necessary

```
ubuntu@ubuntu2404:~/Downloads/code$ gcc fib.c -o fib
ubuntu@ubuntu2404:~/Downloads/code$ javac Fibonacci.java
```

- Make them executable

```
ubuntu@ubuntu2404:~/Downloads/code$ chmod +x fib
ubuntu@ubuntu2404:~/Downloads/code$ sudo chmod a+x fib.sh
ubuntu@ubuntu2404:~/Downloads/code$
```

- Run them

```
ubuntu@ubuntu2404:~/Downloads/code$ ./fib
Fibonacci(18) = 2584
Execution time: 0.03 milliseconds
ubuntu@ubuntu2404:~/Downloads/code$ java Fibonacci
Fibonacci(18) = 2584
Execution time: 0.24 milliseconds
ubuntu@ubuntu2404:~/Downloads/code$ python fib.py
Command 'python' not found, did you mean:
  command 'python3' from deb python3
  command 'python' from deb python-is-python3
ubuntu@ubuntu2404:~/Downloads/code$ python3 fib.py
Fibonacci(18) = 2584
Execution time: 0.26 milliseconds
ubuntu@ubuntu2404:~/Downloads/code$ sudo ./fib.sh
Fibonacci(18) = 2584
Excution time 3307 milliseconds
ubuntu@ubuntu2404:~/Downloads/code$
```

- Which (compiled) source code file performs the calculation the fastest?

Fib.c

Assignment 4.4: Optimize

Take relevant screenshots of the following commands:

a) Figure out which parameters you need to pass to **the gcc** compiler so that the compiler performs a number of optimizations that will ensure that the compiled source code will run faster. **Tip!** The parameters are usually a letter followed by a number. Also read **page 191** of your book, but find a better optimization in the man pages. Please note that Linux is case sensitive.

```
ubuntu@ubuntu2404:~/Downloads/code$ gcc -O2 fib.c -o fib
ubuntu@ubuntu2404:~/Downloads/code$
```

b) Compile **fib.c** again with the optimization parameters

```
ubuntu@ubuntu2404:~/Downloads/code$ gcc -O2 fib.c -o fib_opt
ubuntu@ubuntu2404:~/Downloads/code$
```

c) Run the newly compiled program. Is it true that it now performs the calculation faster?

```
ubuntu@ubuntu2404:~/Downloads/code$ ./fib
Fibonacci(18) = 2584
Execution time: 0.00 milliseconds
ubuntu@ubuntu2404:~/Downloads/code$ ./fib_opt
Fibonacci(18) = 2584
Execution time: 0.01 milliseconds
ubuntu@ubuntu2404:~/Downloads/code$
```

d) Edit the file **runall.sh**, so you can perform all four calculations in a row using this Bash script. So the (compiled/interpreted) C, Java, Python and Bash versions of Fibonacci one after the other.

```
Running C program:
Fibonacci(19) = 4181
Execution time: 0.01 milliseconds


Running Java program:
Fibonacci(19) = 4181
Execution time: 0.28 milliseconds


Running Python program:
Fibonacci(19) = 4181
Execution time: 0.41 milliseconds


Running BASH Script
Fibonacci(19) = 4181
Excution time 5292 milliseconds


ubuntu@ubuntu2404:~/Downloads/code$
```

Assignment 4.5: More ARM Assembly

Like the factorial example, you can also implement the calculation of a power of 2 in assembly. For example you want to calculate $2^4 = 16$. Use iteration to calculate the result. Store the result in r0.

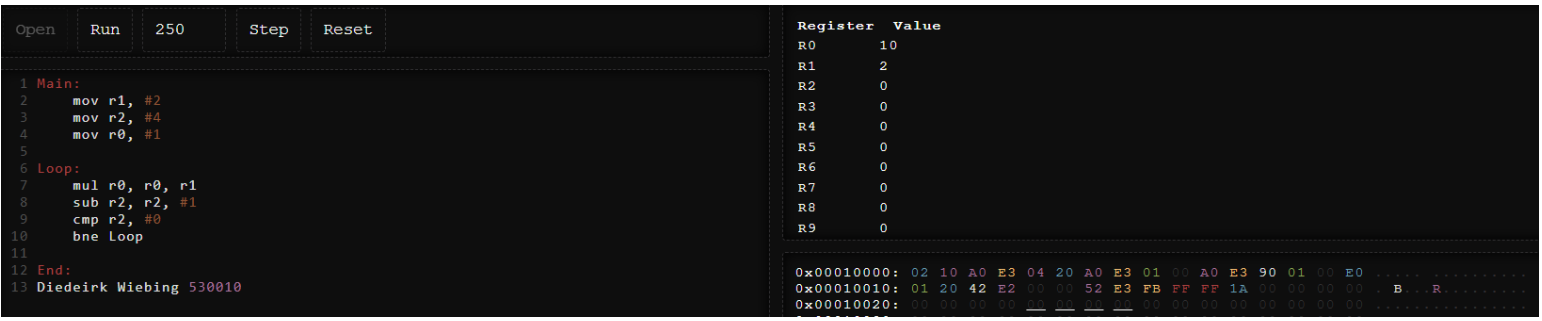
```
Main:
mov r1, #2
mov r2, #4

Loop:

End:
```

Complete the code. See the PowerPoint slides of week 4.

Screenshot of the completed code here.



Ready? Save this file and export it as a pdf file with the name: **week4.pdf**